

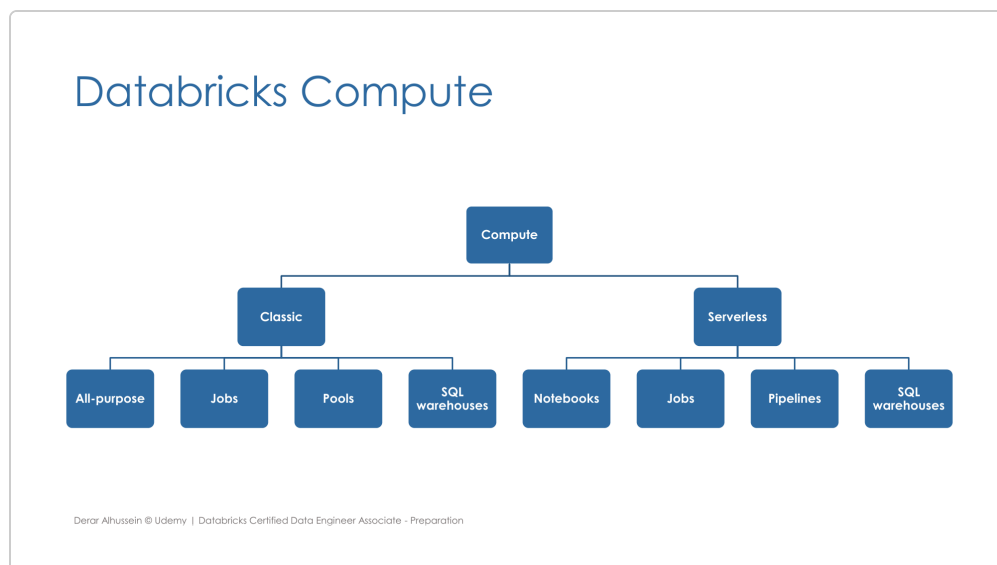
Compute vertieft: Instance-Typen, Pools und Spot-Instances

Section 1 · Databricks Intelligence Plattform · Quelle: Udemy-Kursmaterial „Cluster Best Practices“, ergänzt mit Databricks-Dokumentation

Kapitel 2 dieser Section hat die vier Compute-Typen (All-Purpose, Job, SQL-Warehouse, Serverless) und ihre Kostenlogik bereits vorgestellt, Section 6 Kapitel 6 die Spot-Instance-Konfiguration. Dieses Kapitel ergänzt zwei praktische Bausteine der Cluster-Konfiguration, die dort noch nicht behandelt wurden: die Wahl des richtigen **Instance-Typs** je nach Workload und **Instance Pools** zur Beschleunigung des Cluster-Starts.

1. Classic vs. Serverless: die zwei Grundkategorien

Sämtliche Compute-Typen aus Kapitel 2 lassen sich einer von zwei übergeordneten Kategorien zuordnen: **Classic Compute** (All-Purpose, Job, Pools, SQL-Warehouses – die Infrastruktur wird vom Cloud-Anbieter bereitgestellt und ist konfigurierbar) und **Serverless Compute** (Notebooks, Jobs, Pipelines, SQL-Warehouses – Databricks verwaltet die Infrastruktur vollständig selbst).



Databricks Compute gliedert sich in Classic Compute (konfigurierbare Cloud-Infrastruktur) und Serverless Compute (vollständig von Databricks verwaltet).

2. Instance Family: die passende VM-Kategorie wählen

Bei Classic Compute lässt sich die zugrunde liegende virtuelle Maschine gezielt nach Workload-Typ auswählen. Die falsche Wahl kostet entweder unnötig Geld (überdimensioniert) oder Performance (unterdimensioniert):

VM-Kategorie	Geeignet für
Memory Optimized	Umfangreiches Shuffling/Spilling, Spark-Caching, speicherintensive ML-Workloads
Compute Optimized	Structured-Streaming-Jobs, ELT mit vollständigem Scan ohne Datenwiederverwendung, OPTIMIZE / ZORDER
Storage Optimized	Delta-Caching, Ad-hoc-/interaktive Analysen, ML/DL-Workloads mit Daten-Caching
GPU Optimized	ML/DL-Workloads mit außergewöhnlich hohem Speicherbedarf
General Purpose	Keine speziellen Anforderungen; VACUUM

Instance Family

VM Category	Workload Type
Memory Optimized	• Where a lot of shuffle and spills are involved • When Spark caching is needed • For ML workloads
Compute Optimized	• Structured Streaming jobs • ELT with full scan and no data reuse • To run OPTIMIZE and Z-order Delta commands
Storage Optimized	• To leverage Delta caching • For ad hoc and interactive data analysis • ML and DL workloads with data caching
GPU Optimized	• ML and DL workloads with an exceptionally high memory requirement
General Purpose	• Used in absence of specific requirements • To run VACUUM Delta command

Derar Alhusein © Udemy | Databricks Certified Data Engineer Associate - Preparation

Worker type

Standard_D4ds_v5 16 GB Memory, 4 Cores

- General purpose**
 - Standard_DS3_v2 14 GB Memory, 4 Cores
 - Standard_DS4_v2 28 GB Memory, 8 Cores
 - Standard_DS5_v2 56 GB Memory, 16 Cores
- Memory optimized**
 - Standard_DS12_v2 28 GB Memory, 4 Cores
 - Standard_DS13_v2 56 GB Memory, 8 Cores
- Storage optimized**
 - Standard_L8s_v3 64 GB Memory, 8 Cores
 - Standard_L16s_v3 128 GB Memory, 16 Cores
 - Standard_L32s_v3 256 GB Memory, 32 Cores
- Compute optimized**

Die Wahl der VM-Kategorie richtet sich nach dem Workload-Typ: Memory-, Compute-, Storage- oder GPU-optimiert, oder General Purpose ohne besondere Anforderungen.

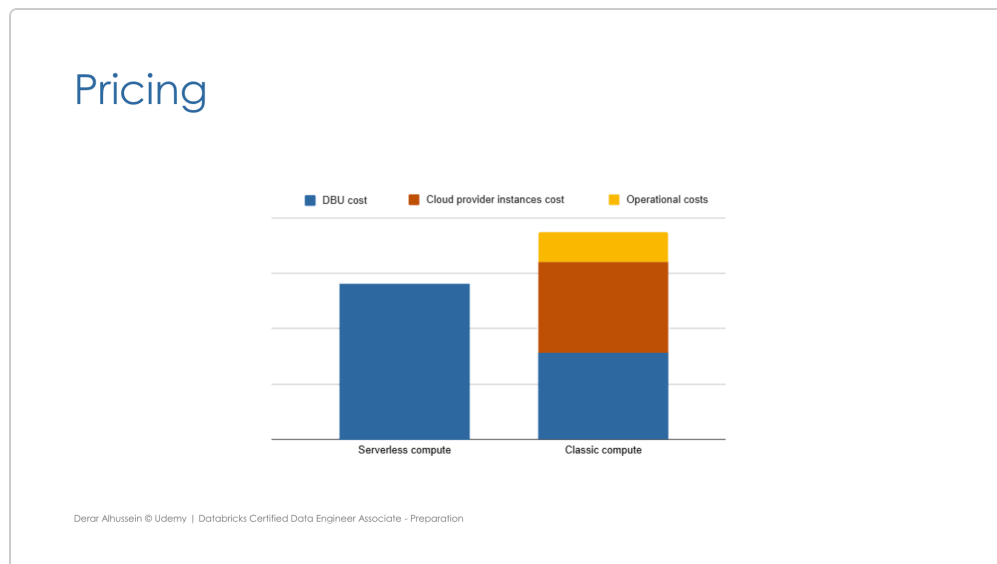
Spot-Instances als weiteres Mittel zur Kostensenkung – inklusive der bewährten Driver-on-Demand/Worker-as-Spot-Konfiguration und konkreter Ersparnis-/Ausfallraten je Instanztyp – behandelt bereits Section 6, Kapitel 6 ausführlich.

3. Instance Pools: schnellerer Cluster-Start

Ein **Instance Pool** ist eine Menge bereits laufender, ungenutzter virtueller Maschinen, die auf ihren Einsatz warten. Wird ein neuer Cluster (oder eine Autoscaling-Erweiterung) aus einem Pool heraus erstellt, entfällt die sonst übliche VM-Boot-Zeit – Cluster-Start und Autoscaling werden dadurch spürbar beschleunigt. Wichtig für die Kostenkalkulation: Databricks selbst berechnet für ungenutzte Instanzen im Pool **keine** DBU-Gebühr, der Cloud-Anbieter stellt diese Instanzen aber weiterhin in Rechnung, da sie technisch im eigenen Cloud-Konto laufen.

4. Pro vs. Serverless SQL-Warehouse

Ergänzend zur Übersicht aus Kapitel 2: Bei **klassischen (Pro) SQL-Warehouses** greift man konkret dann, wenn entweder in der gewählten Cloud-Region **kein Serverless-Warehouse verfügbar** ist, oder wenn ein **benutzerdefiniertes Netzwerk-Setup** benötigt wird – etwa um aus dem SQL-Warehouse heraus Datenbanken oder Event-Busse im eigenen Cloud-Netzwerk oder On-Premises anzubinden. Serverless-Warehouses bleiben für die meisten Fälle (ETL, BI, explorative Analysen) die einfachere und schneller startende Wahl.



Kostenvergleich: Serverless Compute bündelt DBU- und Infrastrukturkosten in einem einzigen Posten; bei Classic Compute kommen Cloud-Anbieter-Instanzkosten und ggf. operative Kosten für die manuelle Cluster-Verwaltung hinzu.

Ergänzt aus der offiziellen Databricks-Dokumentation: Databricks empfiehlt, die Instance-Family-Auswahl regelmäßig anhand des Spark-UI (insbesondere Shuffle-Read/-Write und Spill-Metriken) zu überprüfen und bei Bedarf anzupassen, statt sie einmalig festzulegen – Workload-Charakteristika ändern sich mit wachsendem Datenvolumen. Für Instance Pools gilt zudem: Ein Pool kann eine minimale Anzahl an Idle-Instanzen sowie eine maximale Gesamtkapazität festlegen; Instanzen oberhalb des Minimums werden nach einer konfigurierbaren Leerlaufzeit automatisch beendet, um unnötige Cloud-Kosten zu vermeiden.

Quelle: [Pool configuration reference – Databricks-Dokumentation](#) · [Compute configuration reference – Databricks-Dokumentation](#)

Databricks-Dokumentation (Deutsch) — erstellt aus internem Kursmaterial, ergänzt mit Inhalten der offiziellen Databricks-Dokumentation (docs.databricks.com).